

LAPORAN STRUKTUR DATA

TUGAS PERTEMUAN 3



Dibuat oleh :
Sang Prabu Warku Dhara
25071207847
TI-B

Mata Kuliah :
Struktur Data

Dosen Pengampu :
Reny Fitri Yani, S.T, M.T

PRODI S1 TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS RIAU
T.A.2025/2026

DAFTAR ISI

DAFTAR ISI	i
BAB I PENDAHULUAN	1
1.1 LATAR BELAKANG	1
1.2 TUJUAN	2
1.3 MANFAAT	2
BAB II PEMBAHASAN	3
2.1 Pengertian OOP	3
2.2 Inharetance	4
2.3 Innerclass	5
2.4 Polymorphism	6
2.5 Encapsulation	7
2.6 Kode OOP	8
BAB III PENUTUP	9
3.1 Kesimpulan	9

BAB I

PENDAHULUAN

1.1 LATAR BELAKANG

Perkembangan teknologi informasi yang semakin pesat menuntut adanya metode pemrograman yang lebih terstruktur, efisien, dan mudah dikembangkan. Salah satu paradigma pemrograman yang banyak digunakan saat ini adalah Object-Oriented Programming (OOP). OOP merupakan konsep pemrograman yang berorientasi pada objek, di mana data dan fungsi digabungkan dalam satu kesatuan yang disebut kelas (class).

Bahasa pemrograman Python menjadi salah satu bahasa yang mendukung paradigma OOP secara penuh. Python dikenal sebagai bahasa yang sederhana, mudah dipahami, serta memiliki sintaks yang jelas sehingga cocok digunakan oleh pemula maupun profesional. Dalam Python, konsep OOP seperti class, object, inheritance (pewarisan), encapsulation (enkapsulasi), polymorphism (polimorfisme), dan abstraction (abstraksi) dapat diterapkan dengan mudah.

Penerapan OOP dalam Python sangat penting karena membantu programmer dalam mengelola program berskala besar agar lebih terorganisir dan modular. Dengan menggunakan OOP, kode program dapat digunakan kembali (reusability), lebih mudah dipelihara (maintainability), serta mengurangi duplikasi kode. Oleh karena itu, pemahaman mengenai konsep OOP pada Python menjadi hal yang mendasar bagi setiap individu yang ingin mengembangkan aplikasi atau sistem berbasis perangkat lunak secara efektif dan efisien.

1.2 TUJUAN

Adapun tujuan dari pembahasan Object-Oriented Programming (OOP) pada Python adalah sebagai berikut:

1. Memahami konsep dasar OOP seperti class, object, inheritance, encapsulation, polymorphism, dan abstraction.
2. Mengetahui cara penerapan paradigma OOP dalam bahasa pemrograman Python.
3. Meningkatkan kemampuan dalam menyusun program yang terstruktur, modular, dan mudah dikembangkan.
4. Mampu membuat serta mengelola kelas dan objek untuk menyelesaikan permasalahan dalam pemrograman.
5. Mengoptimalkan penggunaan kembali kode (reusability) dan mempermudah proses pemeliharaan program (maintainability).

1.3 MANFAAT

Adapun manfaat mempelajari Object-Oriented Programming (OOP) pada Python adalah sebagai berikut:

1. Membantu memahami cara membuat program yang lebih terstruktur dan terorganisir.
2. Memudahkan dalam pengembangan program berskala besar karena kode bersifat modular.
3. Mengurangi duplikasi kode melalui konsep reusability (penggunaan kembali kode).
4. Mempermudah proses perawatan dan pengembangan sistem (maintainability).
5. Meningkatkan efisiensi kerja programmer dalam menyelesaikan permasalahan pemrograman.
6. Membantu dalam pembuatan aplikasi yang lebih fleksibel melalui konsep inheritance dan polymorphism.

BAB II

PEMBAHASAN

2.1 Pengertian OOP

Object-Oriented Programming (OOP) adalah paradigma pemrograman yang berorientasi pada objek. Dalam OOP, program dibangun berdasarkan konsep objek yang merupakan representasi dari data dan fungsi yang saling berkaitan. Objek dibuat dari sebuah cetak biru yang disebut class.

Pada Python, OOP digunakan untuk mengelompokkan data dan metode dalam satu struktur sehingga program menjadi lebih terorganisir dan mudah dikelola.

Berikut adalah konsep-konsep utama OOP:

a. Class dan Object

- Class adalah cetak biru atau rancangan untuk membuat objek.
- Object adalah hasil instansi dari class.

2.2 Inheritance

Inheritance memungkinkan suatu class (child class) mewarisi properti dan method dari class lain (parent class).

Syntax dasar:

```
class ChildClass(ParentClass):
```

Contoh :

```
class Person:
    def __init__(self, fname, lname):
        self.firstname = fname
        self.lastname = lname

    def printname(self):
        print(self.firstname, self.lastname)

class Student(Person):
    pass

x = Student("Budi", "Santoso")
x.printname()
```

Penjelasan:

1. Student mewarisi semua atribut dan method dari Person.
2. Keyword pass berarti tidak menambahkan apa pun.
3. Child class bisa menambahkan atau mengubah method.

2.3 Innerclass

Python memungkinkan pembuatan class di dalam class lain (nested class / inner class).

Inner class biasanya digunakan untuk:

1. Mengelompokkan class yang hanya digunakan dalam class utama.
2. Meningkatkan struktur dan keteraturan kode.

Contoh :

```
class Outer:
    def __init__(self):
        self.name = "Outer Class"

    class Inner:
        def message(self):
            return "Ini adalah Inner Class"

obj_inner = Outer.Inner()
print(obj_inner.message())
```

Penjelasan:

1. Inner berada di dalam Outer.
2. Untuk mengaksesnya: Outer.Inner().
3. Inner class biasanya digunakan ketika class tersebut hanya relevan dalam lingkup class luar.

2.4 Polymorphism

Polymorphism berarti banyak bentuk. Dalam OOP, method dengan nama yang sama dapat memiliki fungsi berbeda tergantung class-nya.

Contoh :

```
class Car:
    def move(self):
        print("Drive!")

class Boat:
    def move(self):
        print("Sail!")

class Plane:
    def move(self):
        print("Fly!")

for x in (Car(), Boat(), Plane()):
    x.move()
```

Penjelasan:

1. Semua class memiliki method move().
2. Namun output berbeda.
3. Ini disebut polymorphism karena satu method memiliki banyak bentuk perilaku.

Polymorphism juga bisa terjadi melalui inheritance (method overriding).

2.5 Encapsulation

Encapsulation adalah konsep membungkus data (variabel) dan kode (method/fungsi) dalam satu kelas, serta membatasi akses langsung terhadap data tersebut.

Dalam Python:

1. Variabel biasa → bisa diakses dari luar class
2. Variabel dengan satu garis bawah (`_`) bersifat protected (konvensi).
3. Variabel dengan dua garis bawah (`__`) bersifat private (tidak bisa diakses langsung dari luar class).

Contoh :

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.__age = age # variable privat

    def get_age(self):
        return self.__age

p1 = Person("Andi", 20)
print(p1.name)           # Bisa diakses
print(p1.get_age())      # Akses melalui method
```

Penjelasan:

1. `__age` tidak bisa dipanggil langsung seperti `p1.__age`.
2. Data dilindungi dan hanya bisa diakses melalui method.
3. Tujuannya untuk menjaga keamanan dan integritas data.

2.6 Kode OOP

```
class Spiderman:
    def __init__(self, Jaring, Baju, Topeng):
        self.Jaring = Jaring
        self.Baju = Baju
        self.Topeng = Topeng

    def getPeter(self):
        return self.Jaring

    def getAgus(self):
        return self.Baju

    def getMail(self):
        return self.Topeng

    def ubahtopeng(self, Kairil):
        self.Topeng = Kairil

hero1 = Spiderman("Panjang", "hitam", "hitam")
hero2 = Spiderman("Kuat", "merah", "merah")
hero3 = Spiderman("pendek", "biru", "biru")

print(hero1.getPeter())
print(hero2.getAgus())
print(hero3.getMail())
hero3.ubahtopeng("terserah")
print(hero3.getMail())
```

Penjelasan Kode :

1. `__init__()` digunakan untuk menetapkan nilai pada properti objek
2. Jaring, Baju, Topeng adalah parameter.
3. `self.Jaring`, `self.Baju`, `self.Topeng` adalah atribut (variabel milik objek).
4. Ini adalah fungsi untuk mengembalikan nilai atribut dari objek.

```
def getPeter(self):
    return self.Jaring

def getAgus(self):
    return self.Baju

def getMail(self):
```

```
return self.Topeng
```

5. Ini adalah fungsi untuk mengubah atribut, Kairil adalah atribut baru yang menggantikan Topeng.

```
def ubahTopeng(self, Kairil):  
    self.Topeng = Kairil
```

6. Objek dari class Spiderman

```
hero1 = Spiderman("Panjang", "hitam", "hitam")  
hero2 = Spiderman("Kuat", "merah", "merah")  
hero3 = Spiderman("pendek", "biru", "biru")
```

7. Metode pemanggilan untuk mengubah data objek

```
hero3.ubahTopeng("terserah")  
print(hero3.getMail())
```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan pembahasan yang telah dijelaskan, dapat disimpulkan bahwa Object-Oriented Programming (OOP) merupakan paradigma pemrograman yang berorientasi pada objek, di mana data dan fungsi digabungkan dalam satu kesatuan yang disebut class. Dalam Python, OOP diterapkan melalui konsep utama seperti class, object, encapsulation, inheritance, polymorphism, dan abstraction.

OOP bertujuan untuk membuat program lebih terstruktur, modular, dan mudah dikembangkan. Dengan adanya enkapsulasi, data menjadi lebih aman; dengan pewarisan, kode dapat digunakan kembali; dan dengan polimorfisme, program menjadi lebih fleksibel.

Secara keseluruhan, OOP sangat penting dalam pengembangan perangkat lunak modern karena membantu programmer mengelola program yang kompleks menjadi lebih rapi, efisien, serta mudah dalam proses pemeliharaan dan pengembangannya.